

Lab 8 – MongoDB

Big Data

4 de agosto de 2026

Bien, ¡juguemos con MongoDB! Como la última vez, es importante que copies todos los comandos a un archivo de texto para entregarlos luego a u-cursos.

- Entraremos a MongoDB y habilitaremos la base de datos.
 - Ingresa al servidor de la clase como `uhadoop` y ejecuta `mongo`. Ahora estás en una sesión de MongoDB.
 - Ejecuta `show dbs` para ver las bases de datos disponibles.
 - Ejecuta `use tvdb`. Usaremos esta base de datos.
 - Después ejecuta `db` para ver cuál es la base de datos actual. Ejecuta `show collections` para ver las colecciones en la base de datos actual.
- Ejecutaremos algunas consultas iniciales para ver lo que hay en la colección `series`.
 - Consulta por todos los documentos en la colección `series`.
 - Repite el último paso pero esta vez con los datos indentados (`pretty`).
 - Cuenta el número de documentos en la colección.
 - Crea un arreglo con todos los valores únicos para la llave `"type"` en la colección.
- Ahora agregaremos algunos datos nuevos a la colección. Tomaremos algunos datos de tu elección de la API de TVMaze: <http://www.tvmaze.com/api>.
 - Usa la API para encontrar una serie de televisión *que ya no esté en la colección `series`*. (Para elegir una serie única, puedes buscar para todos los valores de `"name"` en la colección.) Con el nombre de una serie única en mente, encontraremos información en JSON sobre ella.
 - Por ejemplo, <http://api.tvmaze.com/singlesearch/shows?q=the-wire> devolverá información sobre la primera serie encontrada para la búsqueda `"the wire"`. Si prefieres, puedes cambiar el string `the+wire` del URL directamente (reemplazando espacios con `+`).
 - Se puede usar <http://jsonprettyprint.com/> para indentar el JSON, pero no importará la indentación una vez cargado en MongoDB.
 - Observa que la llave `id` en los datos no tiene conflicto con (o nada que ver con) la llave interna `_id` que usa MongoDB. La llave `id` en los datos es como cualquier llave. Pero, más adelante, usaremos ese valor de `id` para encontrar más información sobre la serie; así que vale la pena anotar el valor ahora.
 - Carga los datos como un documento en la colección `series` de MongoDB.
- Es hora de hacer algunas consultas con criterios de selección en la colección `series`. Por el momento, no tienes que usar proyección: deberías devolver los documentos enteros. Observa que puedes cambiar los valores específicos de las llaves (como, ej., `Scripted`) si quieres cambiar los resultados, pero no cambies las llaves mismas (como, ej., `type`) ni la forma de la consulta. Como un consejo final, las llaves y sus valores distinguen minúscula y mayúscula.

- Encuentra todas las series del *type Scripted*.
 - Encuentra todas las series en el *genres Crime* (donde al menos un valor en el arreglo es *Crime*).
 - Encuentra todas las series con el *genres Comedy* y *Adventure*.
 - Encuentra todas las series con un *rating* promedio mayor a 9,3.
 - Encuentra todas las series con un *rating* promedio mayor a 9,3 y con un *genres Crime* o *Comedy*.¹
 - Encuentra todas las series con (exactamente) dos *genres*.
- No siempre queremos ver los documentos completos en los resultados. En lo que sigue, deberías usar las características de proyección de MongoDB.²
- Devuelve (sólo) el *name* y *_id* de las series del *type Scripted*.
 - Devuelve (sólo) el *name* de todas las series del *type Scripted*.
 - Devuelve los documentos enteros de todas las series de *type Scripted* suprimiendo sólo los campos de *language* y *_id*.
 - Devuelve el *name* de cada serie y el último *genres* en su arreglo
- Ahora probaremos algunas características de agregación:
- Ordena todas las series con *language English* (mejores ratings primero) e imprime (sólo) sus *names*.³
 - Cuenta cuántas series hay por *genres* (imprime cada genre con su conteo).
- Probaremos la búsqueda de texto:
- Usa `$text` para buscar documentos en la colección **series** con las palabras clave **sociopathic grandfather**.
 - De hecho, no funcionará para la persona que llegue aquí primero; esa persona tendrá que crear un índice con:
 - `db.series.createIndex( { summary: "text" } )`
 Esa persona puede ver más detalles aquí:⁴
 - <https://docs.mongodb.com/manual/core/index-text/>
 ¡Gracias a esa persona!
 - O si esa persona no desea ayudar a otros, puede borrar el índice cuando haya terminado con la búsqueda:
 - `db.series.getIndexes()` (encontrar el nombre del índice pertinente: `NOMBREDELÍNDICE`)
 - `db.series.dropIndex("NOMBREDELÍNDICE")`
 - Prueba con otra búsqueda de texto.
- La última parte: cargaremos alguna información del equipo de producción de la serie.
- Toma los datos de `http://api.tvmaze.com/shows/[ID]/crew` donde `[ID]` es el identificador de TVMaze (no MongoDB) de la serie que definiste antes. ¡Échale un vistazo! Observa que no menciona la serie; entonces, ¿cómo podemos enlazar la serie con el equipo?

¹“O inclusivo”: debería incluir las series con *Crime* y *Comedy* (entre otras series con uno u otro *genres*).

²No debes usar `distinct` aquí.

³Intenta jugar con el orden de las etapas (*project/sort/match*) y observa la diferencia. ¡Asegúrate de entregar el orden correcto!

⁴Una colección sólo puede tener un índice de texto; pero, ese índice puede incluir múltiples campos y puede tener pesos en campos diferentes para su ranking. Se puede buscar por un campo en particular en este caso.

- Crea una colección *sólo* para el equipo de esa serie. Sube la información previa a la colección (una vez más, no a la colección de `crew` sino a la colección de `crew``SERIESNAME`).<sup>5</sup> Se puede insertar un arreglo directamente sin problema con `db.crew``SERIESNAME``.insert([DATA])`. Pero, hay un límite en el número de caracteres (4096) que puedes usar para un comando en la consola. Si tus datos sobrepasan ese límite, verás un error de sintaxis. Si es así, puedes crear un archivo JSON con los datos en el servidor y usar el siguiente comando para cargarlo:
 - ◊ `mongoimport --db tvdb --collection crewLost --file` `FILENAME``.json --jsonArray`
 Es la forma más estándar de cargar documentos más grandes.⁶
- Encuentra y copia el valor de `ObjectId` de tu serie en la colección `series`. Digamos que este valor es `ObjectId("55555")`.
- Ahora tenemos que anidar una referencia a la serie en cada documento de `crew``SERIESNAME`. Para lograr eso, hay que actualizar/establecer el siguiente valor `crew``SERIESNAME`,<sup>7</sup> donde deberías reemplazar el `ObjectId` "55555" con el de tu serie.
 - ◊ `"tvseries": new ObjectId("55555")`
- Ahora tenemos el identificador (llave foránea) de la serie en cada documento del equipo, podemos copiar el documento desde `crew``SERIESNAME` a `crew` y borrar la colección `crew``SERIESNAME`. Para hacer esto puedes ejecutar:
 - ◊ `db.crew``SERIESNAME``.aggregate([`
 - `{ $match: {} },`
 - `{ $merge: { into: "crew", whenMatched: "keepExisting", whenNotMatched: "insert" } }``])`
 - ◊ `db.crew``SERIESNAME``.drop()`
- Ahora, puedes usar `db.series.aggregate(...)` para hacer un *left-outer-join* con el propósito de anidar los datos del equipo en tu serie. Como indicio, aquí está el comando inicial para encontrar tu serie. Hay que cambiar el `ObjectId` y llenar el resto de la etapa `$lookup` (puedes nombrar el campo "as" como "crew"):
 - ◊ `db.series.aggregate( [ { $match: { _id: new ObjectId("55555") } } ],`
 - `{ $lookup: {` `COMPLETAR` `} } ] )`

- Para salir de la consola de `mongodb`, ingresa `exit`.
- ENTREGA a u-cursos un archivo de texto con todos los comandos ingresados a la consola de `mongodb`.

⁵Si cargamos la información del equipo de varias series juntas, no tendremos ninguna forma de distinguir qué persona trabajó en qué película.

⁶Ojo que el límite de peso de un documento JSON en MongoDB es 16MB; con documentos más grandes habría que, por ejemplo, desanidar algunos documentos y ponerlos en otra colección con una llave foránea; veremos algo parecido ahora.

⁷Otra opción es usar `DBRef` que adicionalmente puede codificar la colección (y opcionalmente la base de datos) del documento. Pero la documentación dice que se prefiere el uso de `ObjectId` directamente dado que es más sencillo: <https://docs.mongodb.com/manual/reference/database-references/>. Una razón para usar un `DBRef` es para referenciar documentos en varias colecciones con una misma llave (como una llave foránea).